

**Stop
Chasing
Better
Models.
Build a
Better
Harness.**

Your LLM Works in Dev. But Fails in Prod.

A powerful model nails a task. A cheaper one fails on the same prompt. Production systems become unpredictable, hard to scale, and expensive to fix.

- ✅ **Model-first thinking:** keep swapping models hoping one sticks.
- ✅ **No structure:** prompts, tools and evals live in separate places.
- ✅ **Zero observability:** can't see token cost, failures or latency.

What is **Harness** Engineering?

Build a structured environment around your LLM instead of tuning the model, you control how it operates.

Prompt Layer

Versioned system prompts stored & tracked in LangSmith

Agent Layer

DeepAgents - task planning, virtual FS & sub-agent spawning

Middleware

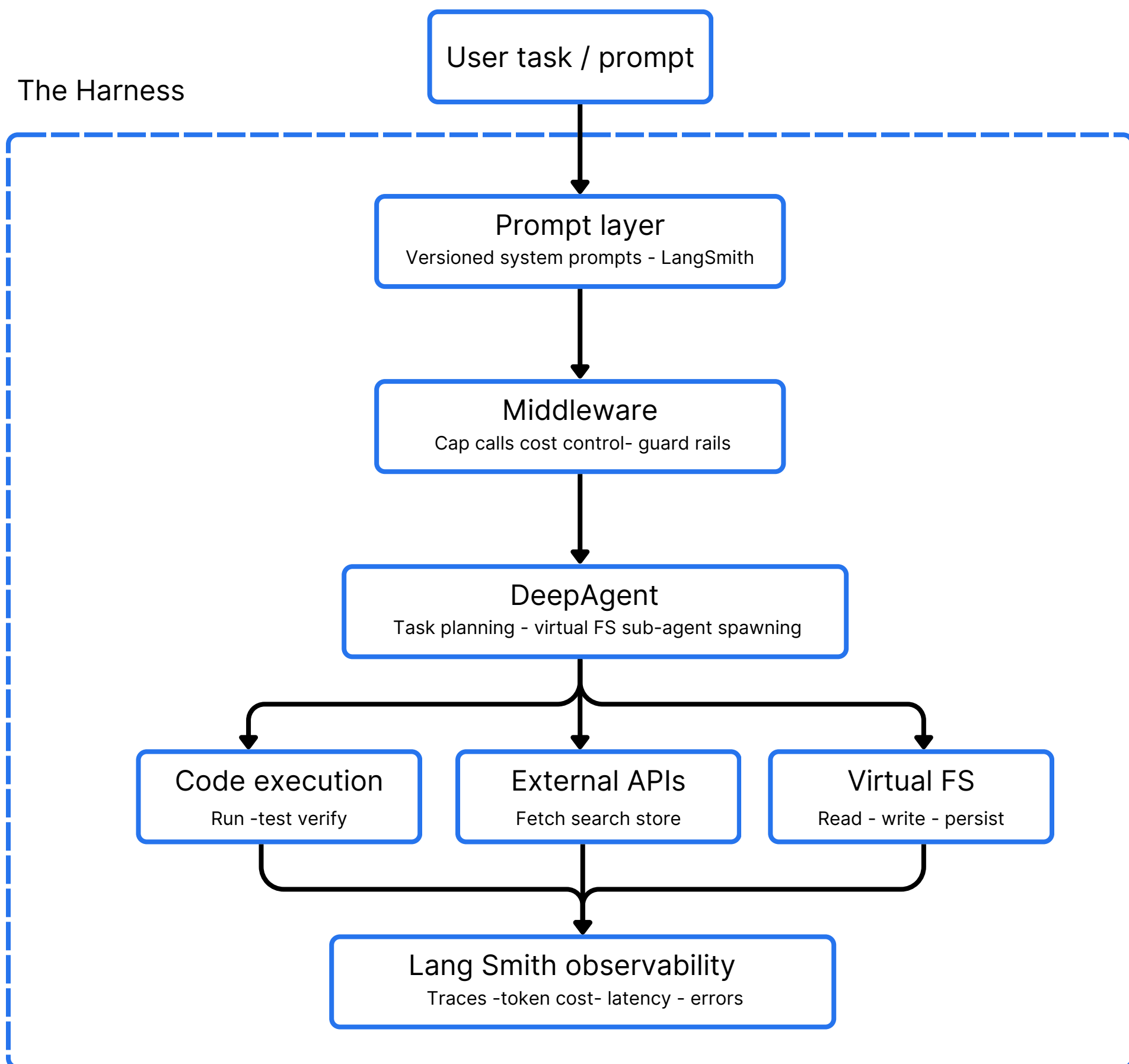
Cap model calls, enforce exit behaviour, extend capabilities

Evaluation

HumanEval pass@1 & pass@k benchmarks for functional correctness

The Full Harness Flow

Build a structured environment around your LLM instead of tuning the model, you control how it operates.



Install, Init & Push Prompts

Build a structured environment around your LLM instead of tuning the model, you control how it operates.



```
# 1. Install deps pip install -qU deepagents langchain-openai
# 2. Set environment keys import os
os.environ['LANGCHAIN_TRACING_V2'] = 'true'
os.environ['LANGSMITH_API_KEY'] = YOUR_KEY
os.environ['OPENAI_API_KEY'] = YOUR_KEY # 3. Push prompt to
LangSmith from langsmith import Client ls = Client()
ls.push_prompt("coding-agent-1", object=prompt)
```

Create a DeepAgent With Middleware



```
from deepagents import create_deep_agent from
langchain.agents.middleware import \
ModelCallLimitMiddleware import init_chat_model model =
init_chat_model("openai:gpt-5-mini") agent =
create_deep_agent( model=model, system_prompt=system_prompt,
middleware=[ ModelCallLimitMiddleware( run_limit=2, # cap
calls exit_behavior="end", # graceful stop ), ], )
```

Measure What Your Agent Actually Does

Don't guess if your harness works - define task-specific tests that match your use case and run them repeatedly.

Task success rate: does the agent complete the job correctly end-to-end?

Consistency: run the same input multiple times -do outputs stay reliable?

Cost & latency: LangSmith traces tokens, time and errors per run automatically.

Prompt comparison: swap prompts in LangSmith and diff results - no model changes needed.

Prompt vs **Harness Engineering**

Prompt Engineering focuses on the input. Harness Engineering controls the environment & constraints.

DeepAgents adds task planning, in-memory file systems and sub-agent spawning out of the box.

Middleware is your circuit-breaker - cap model calls, control costs, prevent runaway agents.

LangSmith gives full traceability: prompts, tokens, cost, errors - all in one place.

Define your **own test cases for your domain** - consistency across runs matters more than any single result.

Resources

Explore the following resources to learn more about Harness Engineering:

Blog

AI Agents

Intermediate

Harness Engineering with LangChain DeepAgents and LangSmith

Learn how Harness Engineering helps build consistent AI systems using LangChain DeepAgents, LangSmith, and HumanEval.



Upcoming DHS Sessions



Harness Engineering for Enterprise AI Agents



Abhishek Kumar
Senior Director Data Science , Publicis S...



The Memory is the Harness: Why Your Moat Isn't the Model



Joinal Ahmed
Senior Architect - AI , Aivar Innovations